

Spark Adaptive Query Execution (AQE) Internals

Prepared By: Kaviraj T U

How Spark Rewrites Your Plan at Runtime

👉 Many engineers tune Spark jobs manually:

- Adjust shuffle partitions
- Change join strategy
- Handle skew manually
- Increase executor memory
- But modern Spark can do something powerful:

👉 It can **change the execution plan at runtime**.

👉 That feature is called **Adaptive Query Execution (AQE)**.

👉 And understanding its internals is a senior-level Spark skill.

What is AQE?

👉 Normally, Spark:

- 1 Builds a logical plan
- 2 Optimizes it
- 3 Creates a physical plan
- 4 Executes it

- That plan is fixed before execution starts.

👉 Problem?

- Spark doesn't know the *real* data distribution until shuffle happens.
- AQE solves this.

👉 It collects runtime statistics and **modifies the physical plan dynamically**.

When Does AQE Kick In?

👉 AQE activates after a shuffle stage completes.

👉 At that moment Spark now knows:

- Actual partition sizes
- Real data distribution
- True join sizes
- Skewed partitions

👉 Using this information, it can re-optimize the next stage.

What AQE Actually Optimizes

1 Dynamic Shuffle Partition Coalescing

👉 Default shuffle partitions:

```
spark.sql.shuffle.partitions=200
```

👉 But what if data is small?

👉 **Without AQE:**

→ 200 tiny partitions

- Too many small tasks
- Scheduling overhead

👉 **With AQE:**

- Spark merges small partitions automatically
- Fewer, optimal-sized tasks

👉 **Result:**

- ✓ Less overhead
 - ✓ Better resource usage
-

2 Dynamic Join Strategy Switching

- 👉 At planning time, Spark may choose:
 - 👉 Sort-Merge Join.
 - 👉 But at runtime it may discover:
 - 👉 One side is actually small enough to broadcast.
 - 👉 AQE can switch:
 - Sort-Merge → Broadcast Join**
 - 👉 Without you changing code.
 - 👉 This can reduce shuffle dramatically.
-

3 Skew Join Optimization

👉 If one partition is much larger:

👉 **Without AQE:**

- One executor overloaded
- Long-running task
- Potential OOM

👉 **With AQE:**

- Spark splits skewed partition
- Processes it in parallel
- Reduces imbalance

👉 This is huge for production workloads.

How to Enable AQE

```
spark.conf.set("spark.sql.adaptive.enabled","true")
```

👉 Other useful configs:

```
spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled","true")
spark.conf.set("spark.sql.adaptive.skewJoin.enabled","true")
```

👉 In Spark 3+, AQE is often enabled by default.

How to Verify AQE is Working

👉 In Spark UI:

- Look at SQL tab
- Check “AdaptiveSparkPlan” in physical plan
- Compare initial plan vs final plan

👉 You’ll see plan changes during execution.

👉 That’s AQE in action.

When AQE Doesn’t Help

👉 AQE is powerful — but not magic.

👉 It won’t fix:

- Poor partitioning strategy
- Massive data skew beyond threshold
- Bad UDF design
- Memory misconfiguration

👉 It optimizes around runtime statistics — not architectural flaws.

Production Insight

👉 **Before AQE:**

- Engineers manually tuned everything.

👉 **After AQE:**

- Spark handles many optimizations automatically.

👉 But...Senior engineers still:

- Inspect execution plans
- Monitor skew behavior
- Validate join strategy
- Tune thresholds carefully

👉 Automation helps.

👉 Understanding still wins.

Final Takeaway

👉 AQE makes Spark adaptive.

👉 But if you don't understand:

- How shuffle works
- How joins are selected
- How partition sizing affects execution

👉 You won't know whether AQE helped — or hid a deeper issue.

👉 Distributed systems are not about writing queries.

👉 They're about understanding execution.

👉 AQE is Spark's way of becoming smarter at runtime.

👉 Your job is to be smarter than the runtime.

Thank You.!

Happy Learning...